Computación Paralela y Distribuída

2026-II

José Fiestas PhD

October 26, 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Unidad 5: Análisis y programación de algoritmos paralelos

Objetivos:

1. Naturalmente (vergonzosamente) algoritmos paralelos
2. Ecuaciones diferenciales parciales
3. Algoritmos de ordenamiento en paralelo
4. Algoritmos de búsqueda en paralelo
5. Algoritmos de grafos en paralelo

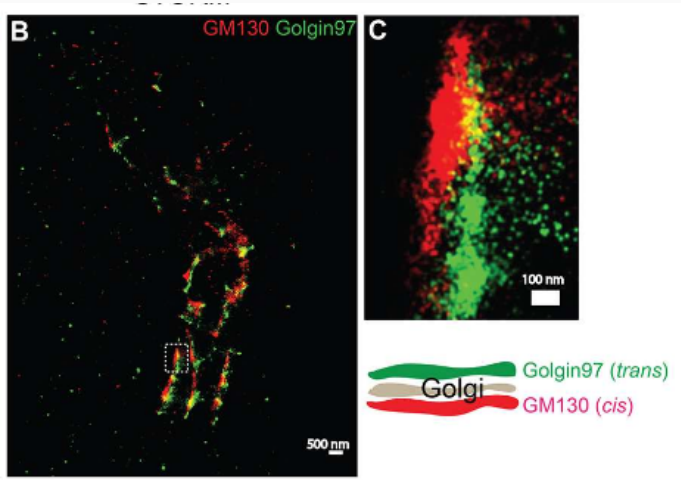
Algoritmos de Ordenamiento

Algoritmos de Ordenamiento

Ordenamiento es uno de los algoritmos fundamentales más usados.

a) **Ordenamiento de proteinas** en la red del Trans-Golgi (Visualization of Protein Sorting at the Trans-Golgi Network and Endosomes Through Super-Resolution Imaging, Huang et.al 2019)

b) **Ordenamiento de capas de renderizado** (The Equalizer Parallel Rendering Framework, Eilemann, 2007). En animaciones se utiliza para priorizar capas donde hay movimiento.



Ordenamiento secuencial: sorting

En C++:

```
void qsort (void* base, size_t num, size_t size,
int (*compar)(const void*,const void*));
```

- **base**: puntero al primer objeto del array a ser ordenada
- **num**: número de elementos del array apuntada por base
- **size**: tamaño en bytes de cada elemento en el array
- **compar**: puntero a una función que compara dos elementos. Con punteros como argumentos, la función define el orden de los elementos retornando <0, 0 o >0

```
 int compar (const void* p1, const void* p2);

int compareMyType (const void * a, const void * b) {
if ( *(MyType*)a < *(MyType*)b ) return -1;
if ( *(MyType*)a == *(MyType*)b ) return 0;
if ( *(MyType*)a > *(MyType*)b )
 return 1;
}
```

Ordenamiento secuencial: sorting

Eficiencia del algoritmo, depende del número de elementos a ser ordenados, basado en la comparación de pares de elementos.

No hay algoritmo más eficiente que O(n log(n))

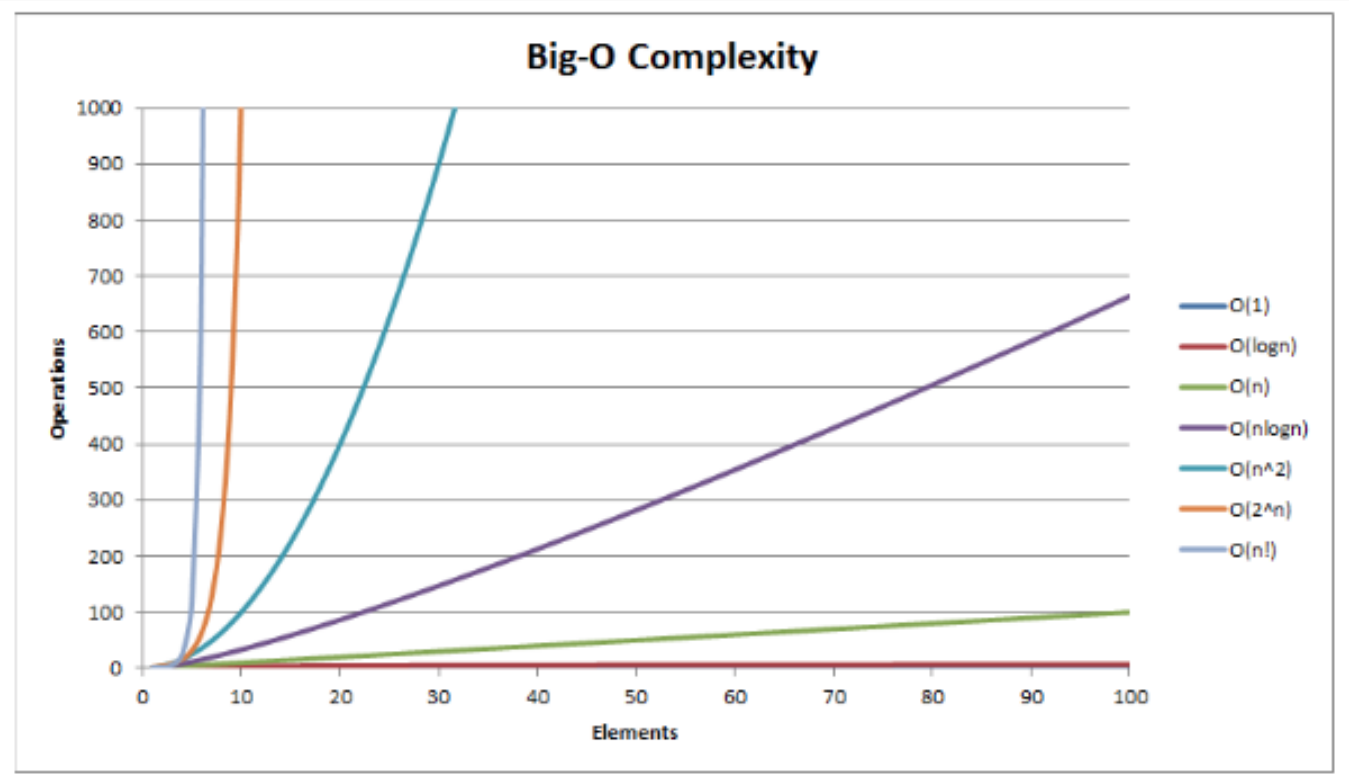
La eficiencia siempre representa casos promedios, a lo que debe añadirse casos extremos: best/worst case

De acuerdo al input, hay casos en que se puede resolver el ordenamiento hasta en O(n), i.e. Bubble sort. Debe considerar la constante en el orden de complejidad O(c*n), la que va a variar de uno a otro algoritmo.

Además:
Espacio de trabajo, hay algoritmos que requieren espacio adicional de trabajo, y por consiguiente mas memoria.
Estabilidad, también puede requerir más tiempo y espacio. Un algoritmo que mantiene el orden de los objetos a través del proceso es más estable.

Ordenamiento secuencial: complejidad

Ordenamiento	Promedio	Mejor	Peor	Estabilidad
Bubblesort	$O(n^2)$	$O(n)$	$O(n^2)$	estable
Selectionsort	$O(n^2)$	$O(n^2)$	$O(n^2)$	estable
Insertionsort	$O(n^2)$	$O(n)$	$O(n^2)$	estable
Heapsort	$O(nlog(n))$	$O(nlog(n))$	$O(nlog(n))$	inestable
Mergesort	$O(nlog(n))$	$O(nlog(n))$	$O(nlog(n))$	estable
Quicksort	$O(nlog(n))$	$O(nlog(n))$	$O(n^2)$	estable



Ordenamiento en paralelo

Caso: p=n

- El mejor algoritmo de ordenamiento secuencial tiene un costo O(n log n). Modelo basado en comparaciones
$T_{seq} = T_{compr}\, n \log n + O(n)$
- Con un **algoritmo paralelo**, utilizando n procesos, se obtiene un costo de O(log n)* $T_{merging}$

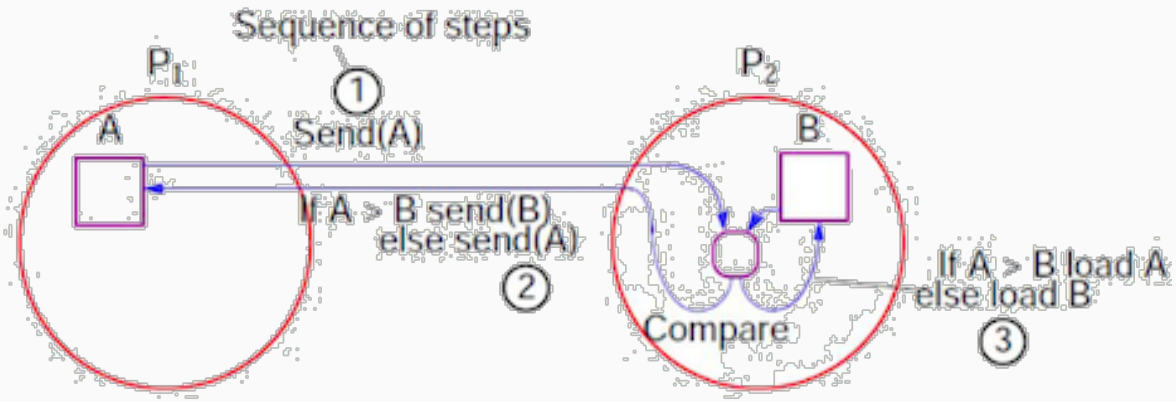
Se puede demostrar que un algoritmo que usa un merging óptimo se resuelve en O(log n) [2] (*Ch.4*)

Ordenamiento en paralelo: operaciones básicas

Operación básica de **comparación** de valores e **intercambio** de sus posiciones en la secuencia.

Caso1:

P1 envía A a P_2. Este compara B y A, y envía el mínimo de A y B a P_1

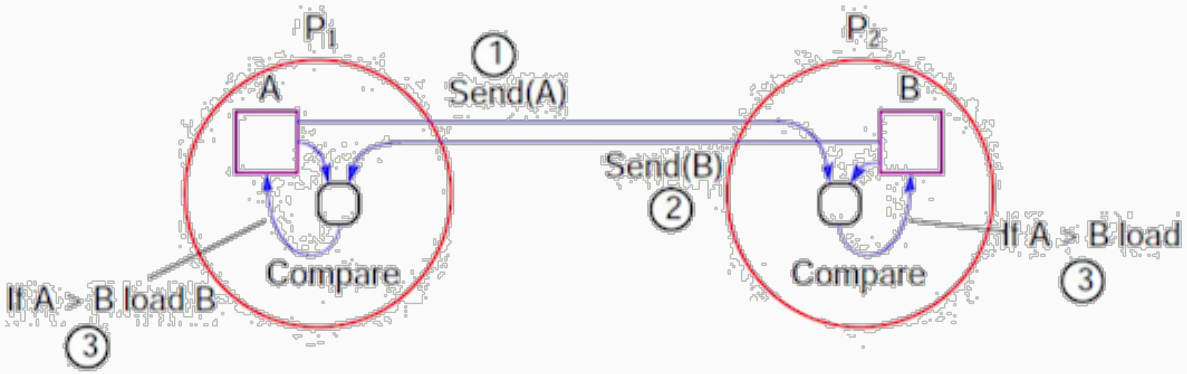


Ordenamiento en paralelo: operaciones básicas

comparación e **intercambio** en paralelo

Caso2:

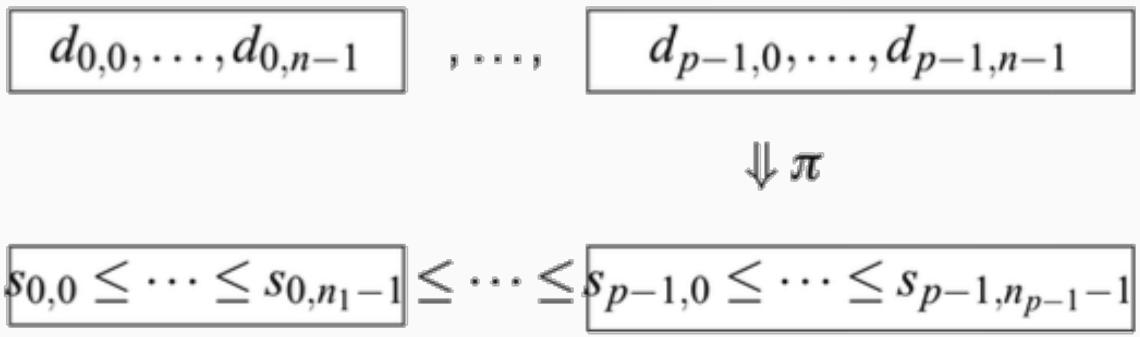
P_1 envía A a P_2. P_2 envía B a P_1. P_1 calcula A=min(A,B), P_2 calcula B=max(A,B)



Ordenamiento en paralelo

Caso: p<n

n elementos, p procesos, n/p elementos por proceso



$$T_{par} = O(\frac{n}{p} log(\frac{n}{p})) + T_{comm}$$

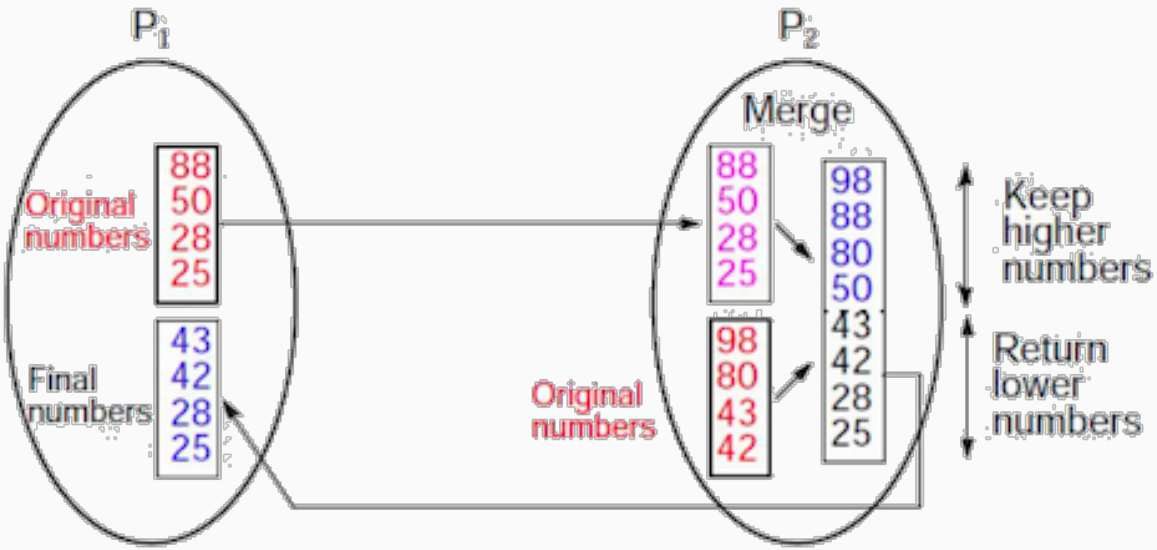
Ordenamiento en paralelo: operaciones básicas

p<n:

n números y p procesos

n/p números en cada proceso

Caso1:



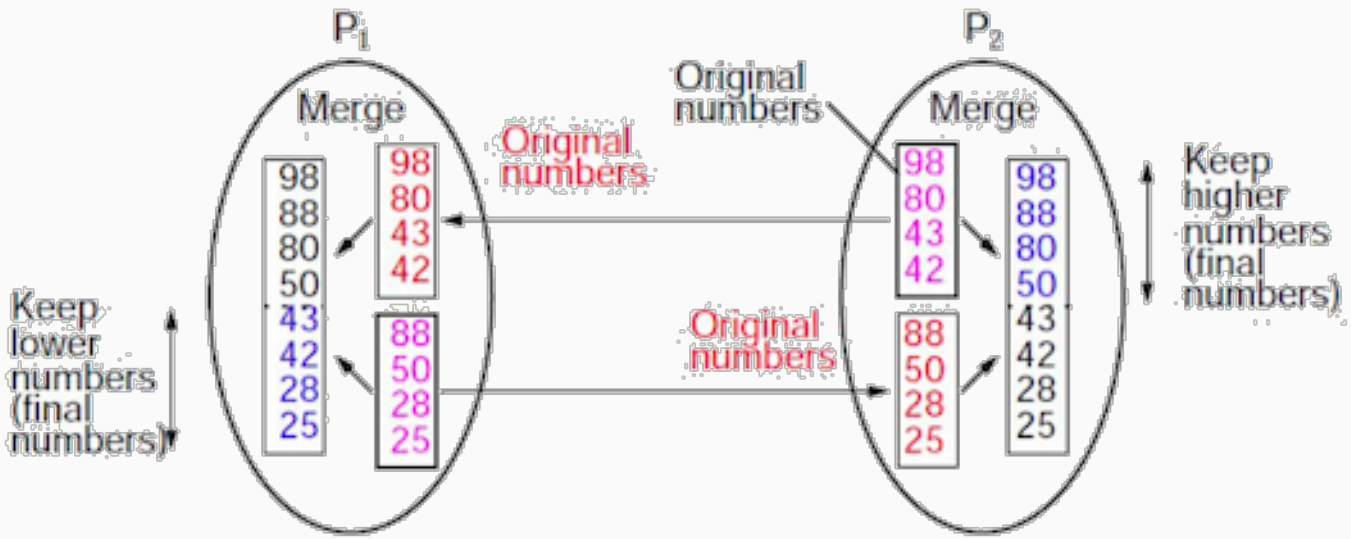
Sorting en paralelo: operaciones básicas

p<n:

n números y p procesos

n/p números en cada proceso

Caso2:



Ordenamiento por ranking en paralelo: algoritmo 1

Método rápido, poco eficiente

n elementos, n^2 procesos

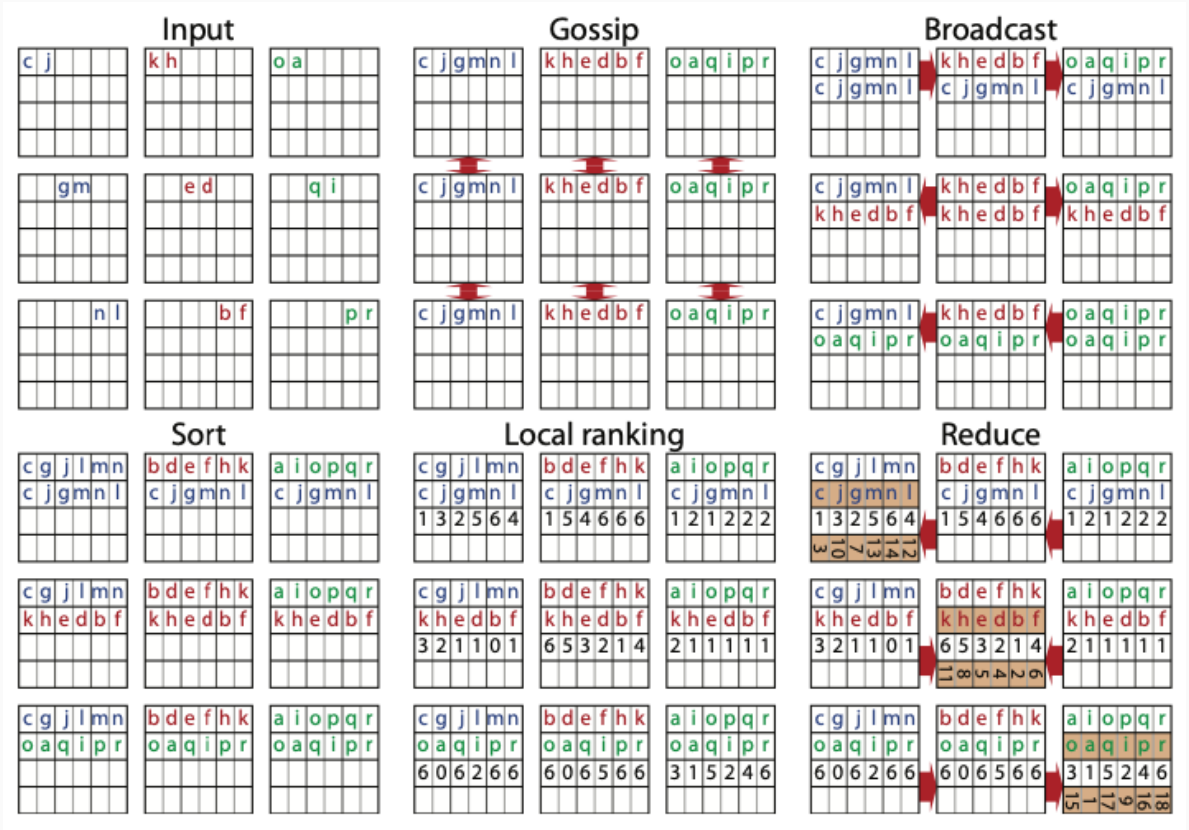
input: A[1,. . . ,n]
output: M[1,. . . ,n]
forall (i,j) $\in \{1, \ldots, n\}^2$ **pardo**
$B[i,j] := A[i] \geq A[j]$
forall $i \in \{$ 1,. . . ,n$\}$ **pardo**
$M[i] = \sum_{j=1}^{n} B[i,j]$

$T_\infty = O(\log n)$

$W = O(n^2 + n \log n)$

```
i   A    B 1   2   3   4   5 <- j   M
1   3      1   0   1   0   1         1
2   5      1   1   1   0   1         1
3   2      0   0   1   0   1         1
4   8      1   1   1   1   1         1
5   1      0   0   0   0   1         1
    A      3   5   2   8   1
-------------------------------------
i   A    B 1   2   3   4   5 <- j   M
1   3      1   0   1   0   1         3
2   5      1   1   1   0   1         4
3   2      0   0   1   0   1         2
4   8      1   1   1   1   1         5
5   1      0   0   0   0   1         1
```

Ordenamiento por ranking en paralelo: algoritmo 2



Ordenamiento por ranking en paralelo: algoritmo 2

n elementos, p procesos, $\frac{n}{\sqrt{p}}$ elementos por proceso

input: **d**[1,. . . ,n]
output: ranking de $\mathbf{d}_{(i,j),k}$ almacenado en $\mathbf{b'}_{(in+k)}$ en el proceso (j,j)

Gossip el input por columna, i.e. p(i,j) recibe **a**:= $a_{1,j}, \ldots, a_{\sqrt{p},j}$
Broadcast el input por fila, i.e. p(i,j) recibe **a'**:= $a_{i,1}, \ldots, a_{i,N}$
sort a
for k:=1, p **do** b_k=ranking de a'_k en **a**
Reduce b por fila, i.e. p(i,i) recibe **b'**:= $\sum_{j=1}^{\sqrt{p}} b(i,j)$

$$T_p(n,p) = O(\log \sqrt{p}(\alpha + \beta(\frac{n}{\sqrt{p}})) + T_{compr}\frac{n \log n}{\sqrt{p}})$$

Ordenamiento por ranking en paralelo: algoritmo 2

$$T_p(n,p) = O(\log\sqrt{p}(\alpha + \beta(\frac{n}{\sqrt{p}})) + T_{compr}\frac{n\log n}{\sqrt{p}})$$

Cuantificando:
p=1024, $\alpha = 10^{-5}s$, $\beta = 10^{-8}s$, $T_{compr} = 10^{-8}s$, $n/p = 32$

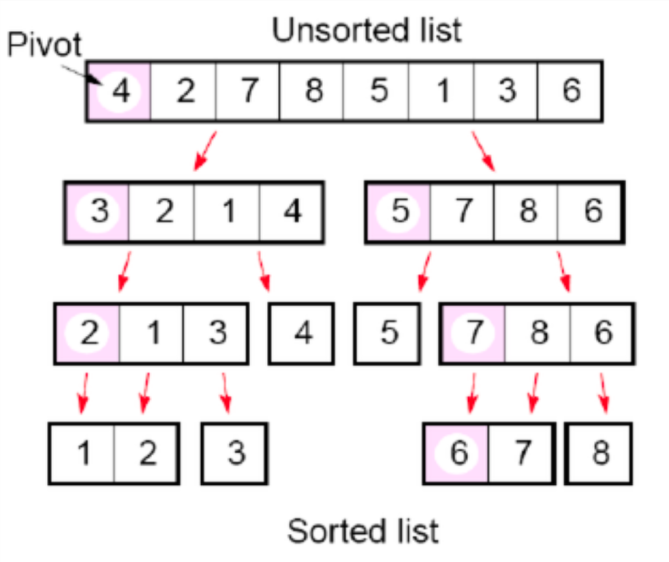
$$T_p(n,p) = k_1 * \log\sqrt{p}(\alpha + \beta(\frac{n}{\sqrt{p}})) + k_2 * T_{compr}\frac{n\log n}{\sqrt{p}})$$

$$\approx k_1 * 0000357 + k_2 * 0.0000011s$$

$k_1, k_2 \in \mathbb{R}$

Ordenamiento rápido (quicksort)

Algoritmo recursivo, uno de los más rápidos y simples algoritmos de ordenamiento. Funciona según el principio de divide y conquista. Complejidad con n elementos es O(n logn) o O(n^2)



Primero se divide la secuencia en elementos menores y mayores que el pivot. Y se llama a la función en forma recursiva.

Quicksort secuencial

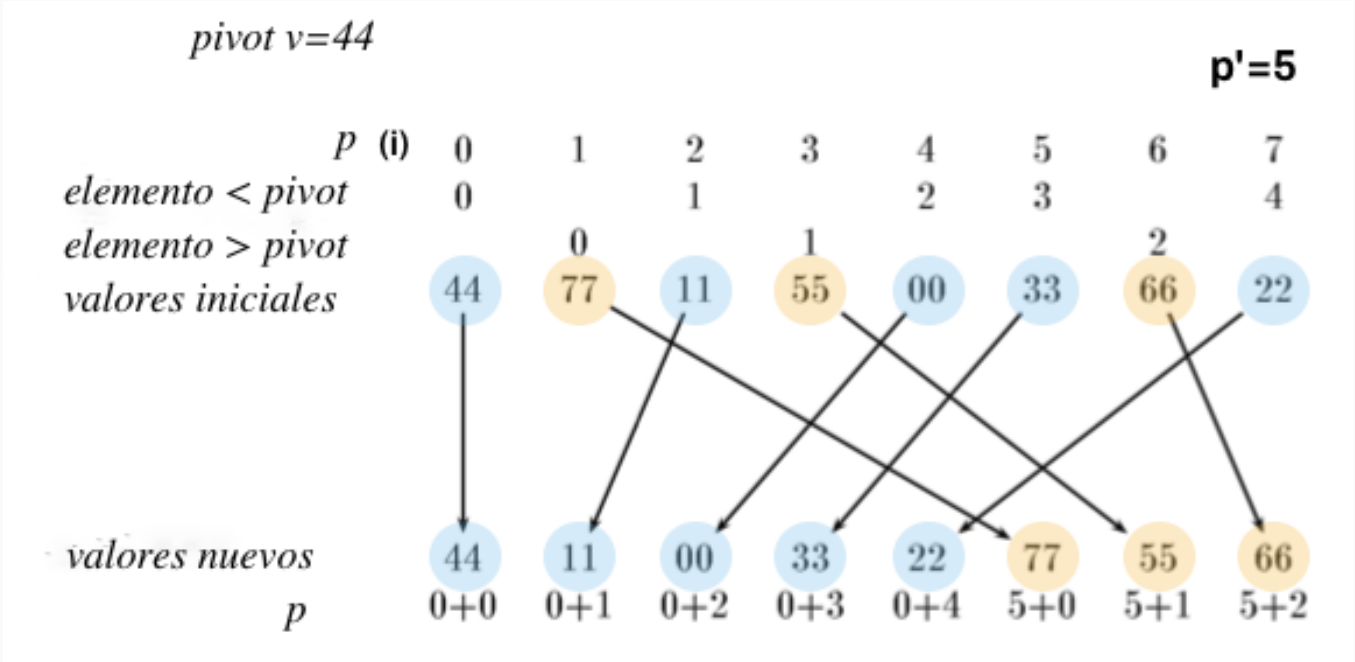
método QS(d[],p)

Entrada: array d de p elementos

Salida: array d ordenada

```
if (p==1) return
k = selecciona_pivot(d,v)
//distribuye elementos en d tal que:
// $d_0 < ... < d_k=v < ... d_{k+1} < . . . < d_{p-1}$
QS([ d_0, . . . , d_{k-1}],k) \\
QS([ d_{k+1}, . . . , d_{p-1}],p-k-1)
```

Quicksort en paralelo (n=p)



El algoritmo de Peter Sanders es el mejor quicksort paralelo de propósito general [4], [1]

Quicksort en paralelo: PRAM + comunicación (n=p)

QS(d,i,p)

Entrada: valor del elemento (d) almacenado en el proceso i, de un total de p procesos

Salida: array ordenada en el maestro

---

if(p=1) **return**
j:= random entre 0 y p-1
v:= d_j // pivot
Broadcast v a todos los procesos para comparación
flag:=d $\leq$ v
j:= $\sum_{k=0}^{i} flag_k$ // suma de prefijos
p' := j_{p-1} // evaluado hasta (p)
Broadcast p' a todos los procesos para comparación
if(flag) **send** d to j-1
else send d to p'+i-j
receive d
if (i $<$ p') {
// i pertenece a la partición izquierda
QS(d,i,p') }
else {
// i pertenece a la partición derecha
QS(d,i-p',p-p') }

Quicksort en paralelo (n=p)

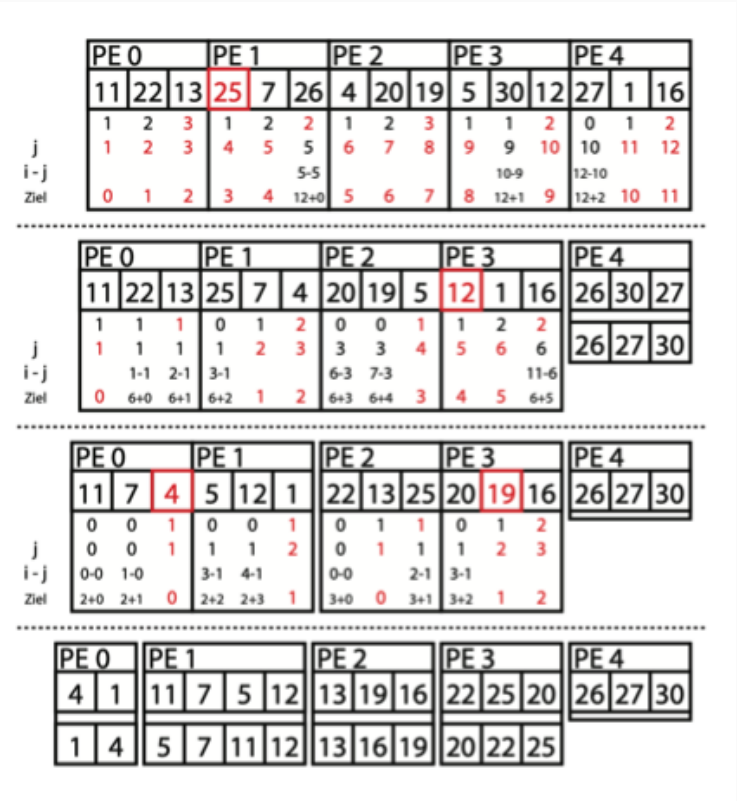
Los procesos se separan en particiones. En una partición, el proceso j contiene el pivot, el cual es enviado a todos los demás procesos (**Broadcast**). Se utiliza luego una **suma de prefijos** para numerar procesos con elementos menores/mayores que el pivot. Se define p' como el proceso con el pivot y se envia al resto de procesos (**Broadcast**). Los primeros p'-1 procesos contienen los p'-1 elementos menores que el pivot. Esta partición se vuelve a dividir en dos nuevas particiones, donde se aplica de nuevo el algoritmo en forma recursiva. I.e.

$$T(p) = T(p/2) + log(p)$$

$$\therefore T_p(n,p) \in O(log^2(p))$$

Quicksort en paralelo (n>p)

En caso de n>>1, cada proceso p recibe n/p elementos y los ordena en un tiempo $O(\frac{n}{p}\log(\frac{n}{p}))$. Estos, a su vez, son transportados entre procesos en un tiempo $O(\log p(\alpha + \frac{n}{p}\beta))$. En PRAM, obtuvimos span $O(\log^2 p)$. Por lo tanto:

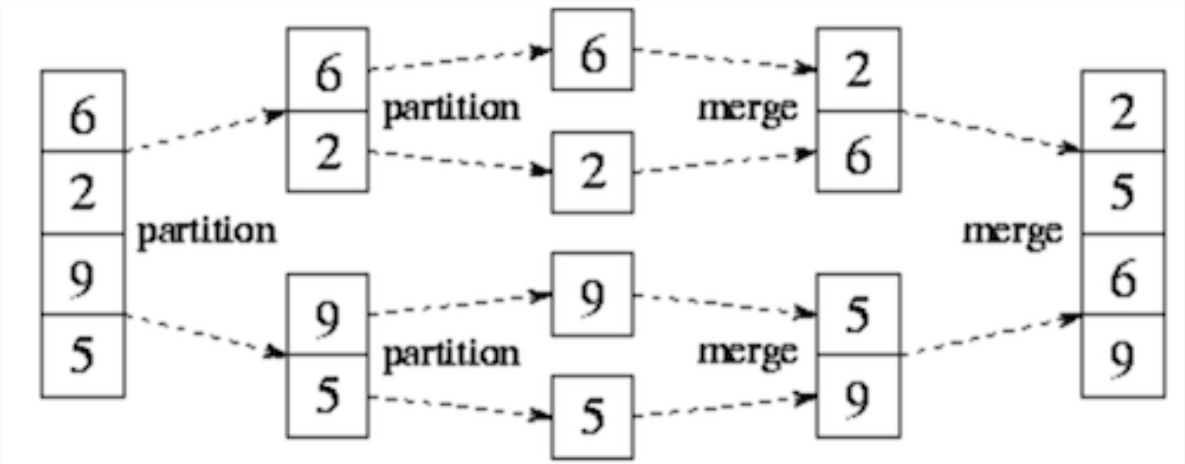


$$T_p(n,p) = O(\frac{n\log n}{p} + \log p(\alpha + \frac{n}{p}\beta) + \alpha\log^2 p)$$

Mergesort

Método:

- Si A=$(a_1, a_2, ..., a_r)$ y B=$(b_1, b_2, \ldots b_s)$ son dos secuencias de números ordenadas en orden ascendente
- Se busca mezclar A y B, es decir, generar una secuencia C=$(c_1, c_2, \ldots, c_m)$, donde m=r+s
- C también será una secuencia ordenada ascendente
- Se divide la lista de n elementosen n sub-listas (cada una con un elemento)
- Se fusionan sublistas para crear nuevas sublistas ordenadas, hasta que solo queda una sublista

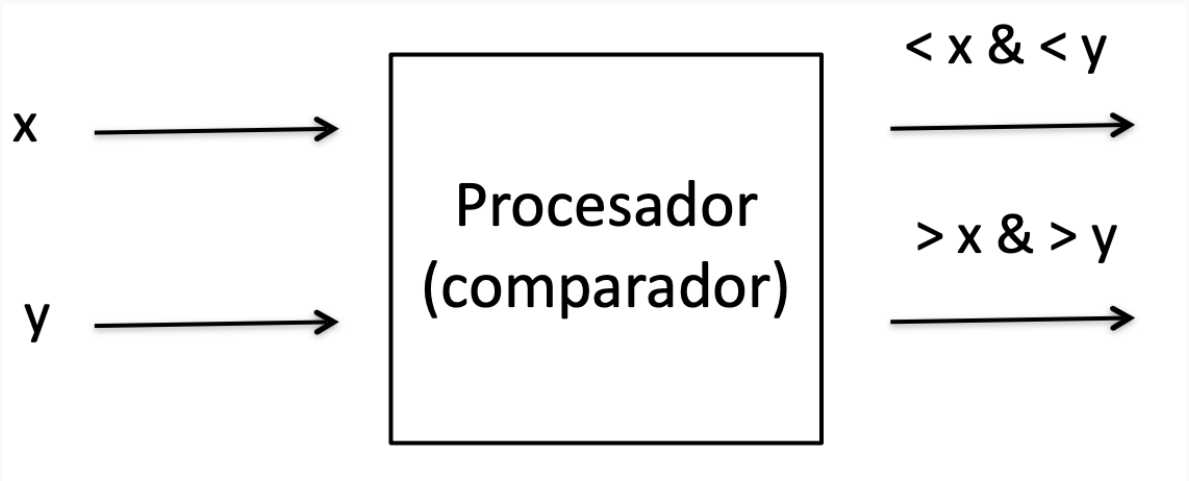


Mergesort

Supuestos:

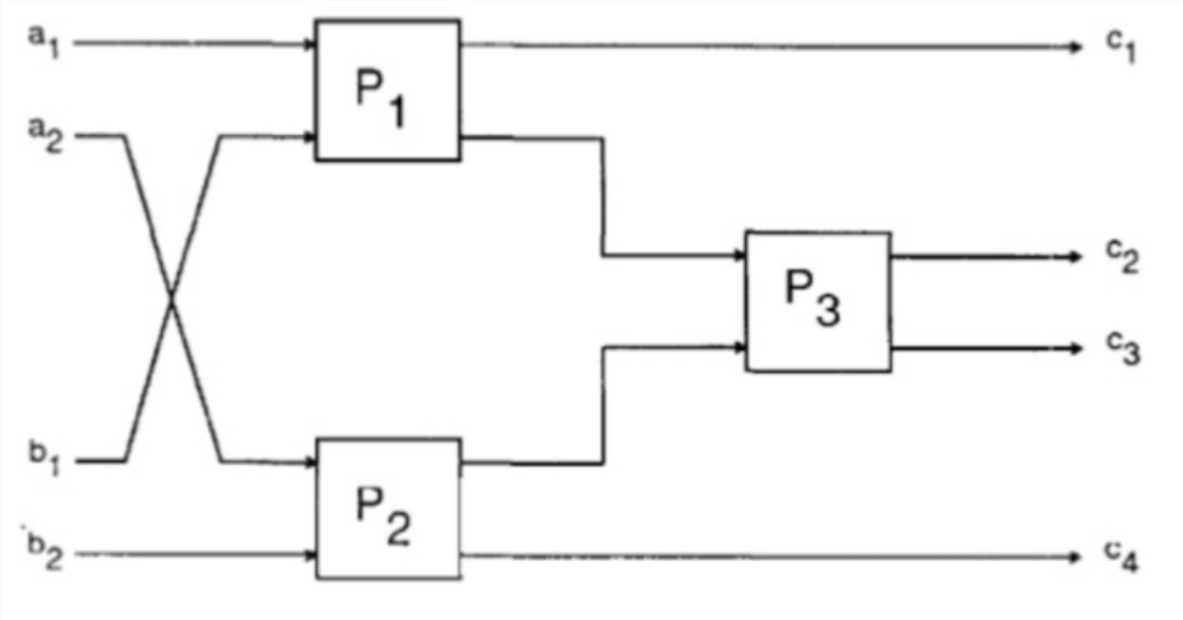
- Ambas secuencias son del mismo tamaño. $r = s = n > 1$
- n es una potencia de 2

Si n=1, el procesador tiene como salida una secuencia ordenada.



Mergesort

Si n=2, las dos secuencias A=(a_1, a_2) y B=(b_1, b_2), se ordenarán de acuerdo al siguiente diagrama

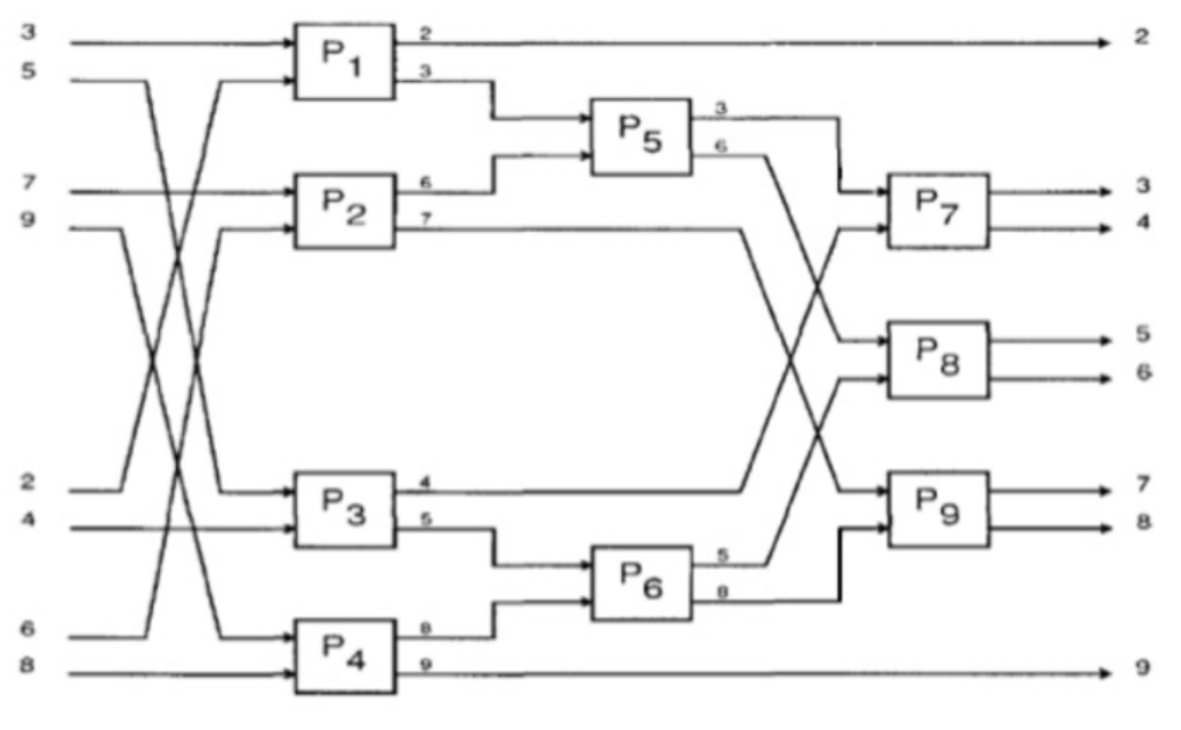


El proceso P_1 compara el menor elemento de A con el menor elemento de B, resultando en el menor elemento de c, i.e. c_1 El proceso P_2 hace lo mismo, resultado en c_4

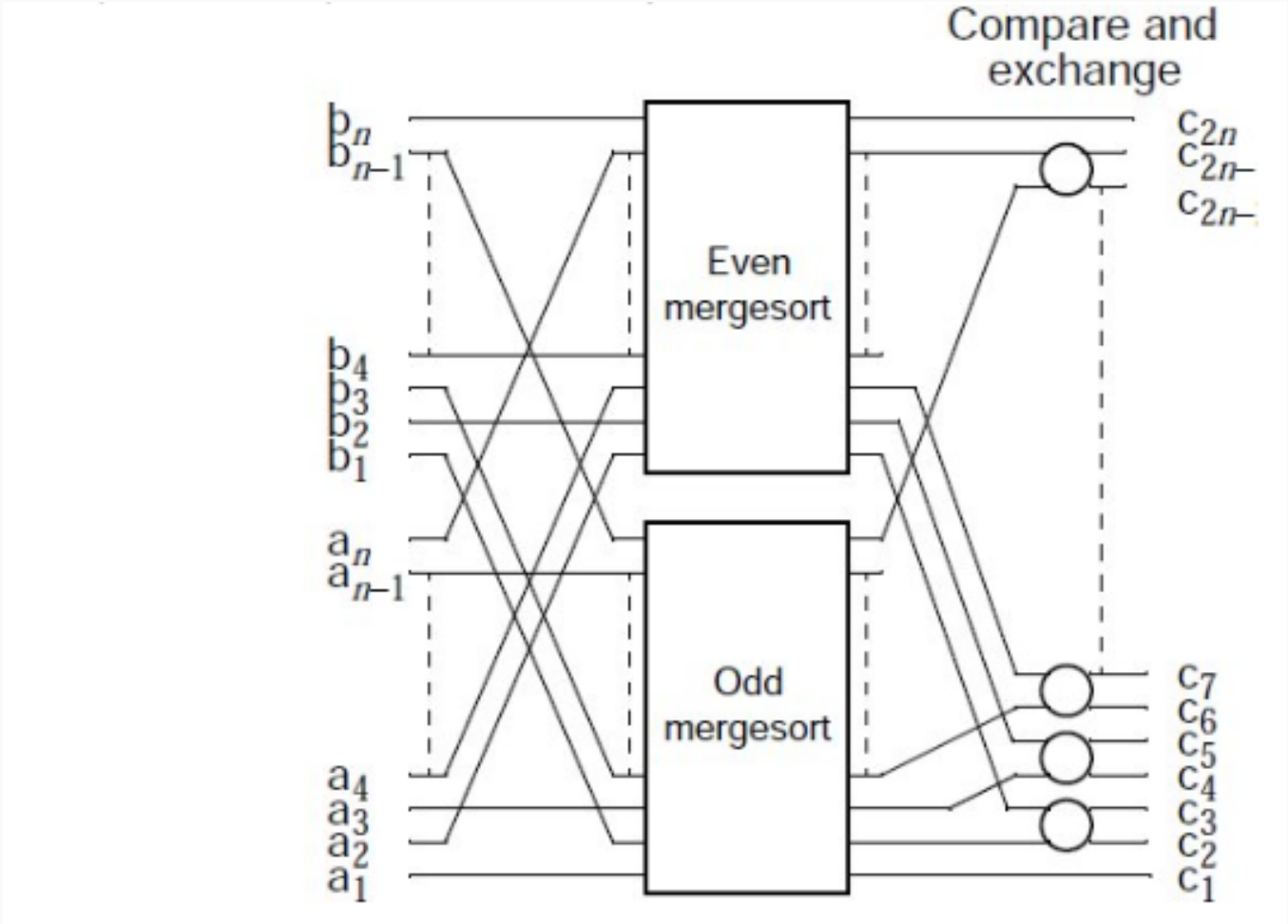
El proceso P_3 hace lo mismo para los valores intermedios

Mergesort

Dos subgrupos de 4 elementos c/u
A={3,5,7,9}, B={2,4,6,8}



Odd-Even Mergesort (de Batcher)



Odd-Even Mergesort (de Batcher)

Algorithm 1: OddEvenMergeSort(A, N)

Entrada: Arreglo A de tamaño N

Salida: Arreglo A ordenado

if $N > 1$ **then**

$A_L \leftarrow \text{OddEvenMergeSort}(A_L, N/2)$: ||

$A_R \leftarrow \text{OddEvenMergeSort}(A_R, N/2)$;

$A \leftarrow \text{OddEvenMerge}(A_L, A_R, N/2)$;

$$T_\infty(n) = T_\infty(n/2) + O(\log n)$$
$$T_\infty(n) = O(\log^2 n)$$

Odd-Even Mergesort (de Batcher)

Algorithm 2: OddEvenMerge(A, B, n)

Entrada: Secuencias ordenadas A y B de tamaño n

Salida: Secuencia ordenada C de tamaño $2n$

if $n = 1$ **then**

  $C \leftarrow \text{CompareExchange}(A[1], B[1])$;

else

  $D \leftarrow \text{OddEvenMerge}(A_I, B_I, n/2)$ ||

  $E \leftarrow \text{OddEvenMerge}(A_P, B_P, n/2)$;

  $C[1] \leftarrow D[1]$; $C[2n] \leftarrow E[n]$;

  for $i \leftarrow 1$ **to** $n - 1$ **pardo**

    $C[2i] \leftarrow \min(D[i + 1], E[i])$;

    $C[2i + 1] \leftarrow \max(D[i + 1], E[i])$;

Odd-Even Mergesort (de Batcher)

Algoritmo recursivo:

Los elementos impares de A y B: $a_1, a_3, a_5, ..., a_{n-1}$, y $b_1, b_3, b_5, ..., b_{n-1}$ se mezclan utilizando subgrupos $(\frac{n}{2}, \frac{n}{2})$ para producir una secuencia $d_1, d_2, ..., d_n$

Simultáneamiente, los elementos pares de A y B : $a_2, a_4, a_6, ..., a_n$, y $b_2, b_4, b_6, ..., b_n$ se mezclan utilizando subgrupos $(\frac{n}{2}, \frac{n}{2})$ para producir una secuencia $e_1, e_2, ..., e_n$

La secuencia final $c_1, c_2, ..., c_{2n}$ se obtiene de $c_1 = d_1$, $c_{2n} = e_n$, $c_{2i} = min(d_{i+1}, e_i)$, y $c_{2i+1} = max(d_{i+1}, e_i)$, para i=1,2,. . . ,n-1

Cada subgrupo $(\frac{n}{2}, \frac{n}{2})$ se construye recursivamente utilizando dos $(\frac{n}{4}, \frac{n}{4})$ subgrupos

Odd-Even Mergesort (de Batcher)

La complejidad de Mergesort en paralelo se muestra en la tabla (abajo)

	Span	**Trabajo**	**Referencia**
Odd-Even Mergesort	$O(\log^2 n)$	$O(n \log n)$	
Two-way-merge	$O(\log n \log \log n)$	$O(n \log n)$	[2] (Ch.4.3.1)

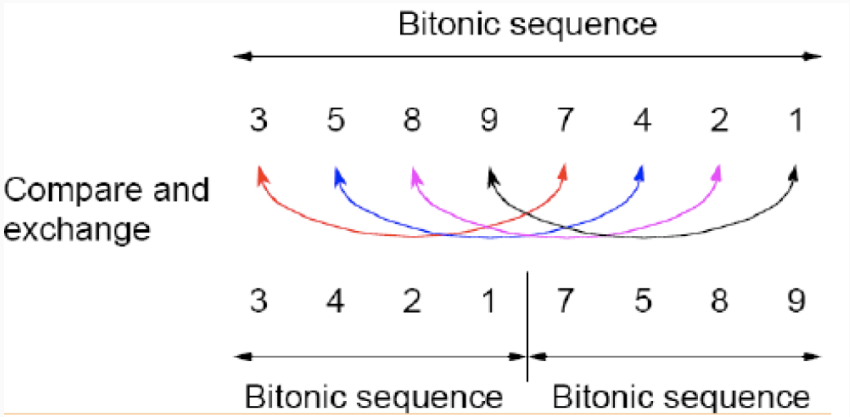
Se puede realizar en un modelo CREW

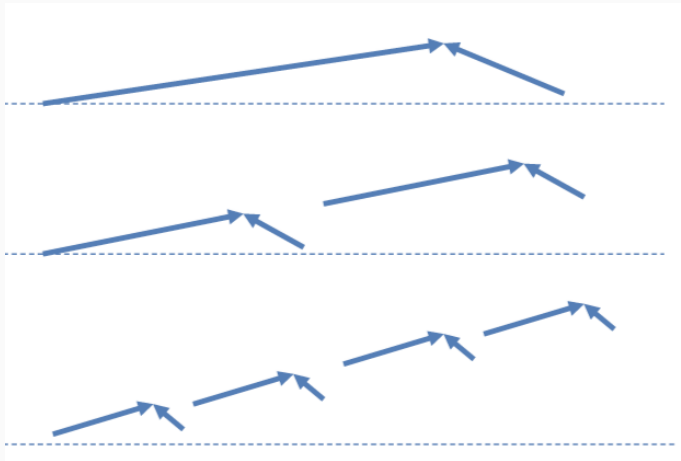
Ordenamiento bitónico

Una secuencia bitónica, consiste en dos secuencias. Una creciente y una decreciente

$$a_0 < a_1 < a_2, \ldots, a_{i-1} < a_i > a_{i+1}, \ldots, a_{n-2} > a_{n-1}$$

Si se realiza una operación de comparación e intercambio de a_i con $a_{i+n/2}$, para todo i, se obtendrán dos secuencias bitónicas, donde los elementos de una secuencia serán menores que los elementos de la otra.





Recursivamente, cada secuencia bitónica final consiste de solo dos elementos

Ordenamiento bitónico (algoritmo)

Algorithm 3: BitonicSort(A, l, n, dir)

Entrada: Arreglo A; posición inicial l; tamaño n; dirección dir

if $n > 1$ **then**

  $m \leftarrow n/2$

    BitonicSort(A, l, m, ASC) ||

    BitonicSort($A, l + m, m, DESC$)

  BitonicMerge(A, l, n, dir)

$$T_\infty(n) = T_\infty(n/2) + O(\log n)$$
$$T_\infty(n) = O(\log^2 n)$$

Ordenamiento bitónico (algoritmo)

Algorithm 4: BitonicMerge(A, l, n, dir)

Entrada: Secuencia bitónica $A[l : l + n]$, dirección dir

if $n > 1$ **then**

  $m \leftarrow n/2$

  for $i \leftarrow 0$ **to** $m - 1$ **pardo**

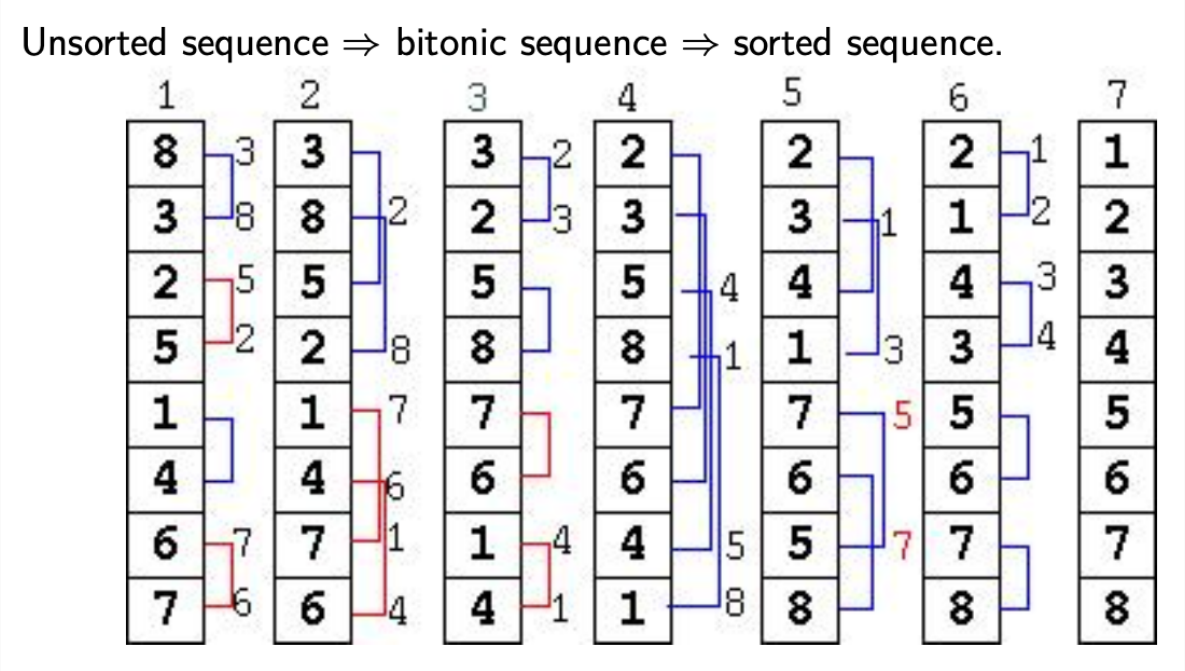
    CompareExchange($A[l + i], A[l + i + m], dir$)

    BitonicMerge(A, l, m, dir) ||

    BitonicMerge($A, l + m, m, dir$)

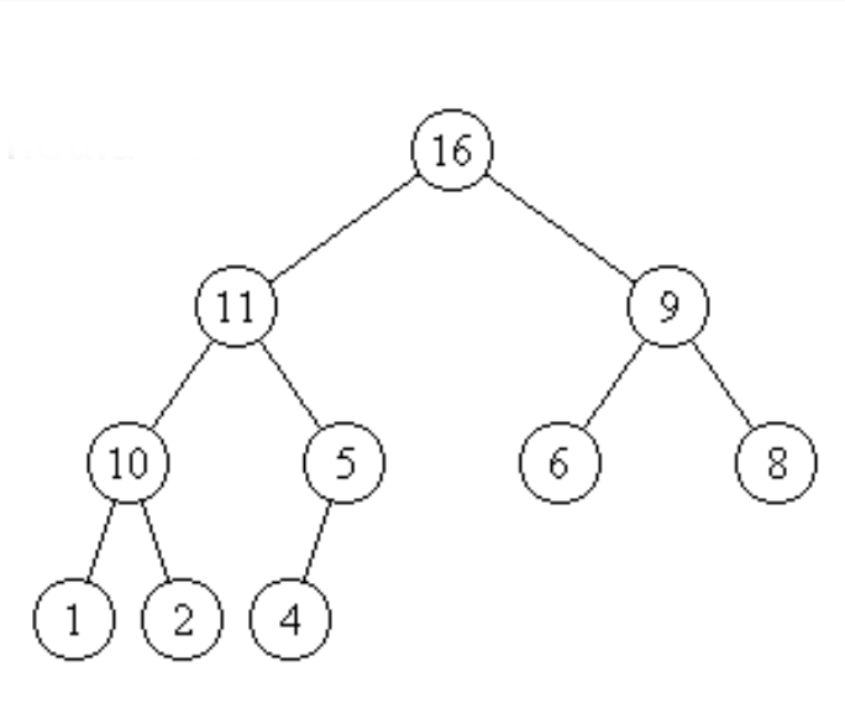
Ordenamiento bitónico

Se parte de una secuencia bitónica, partiendo de la lista inicial de elementos desordenados. Ya que un par de números son bitónicos, se puede particionar la lista inicial en pares, y unirlos consecutivamente en listas mas grandes.



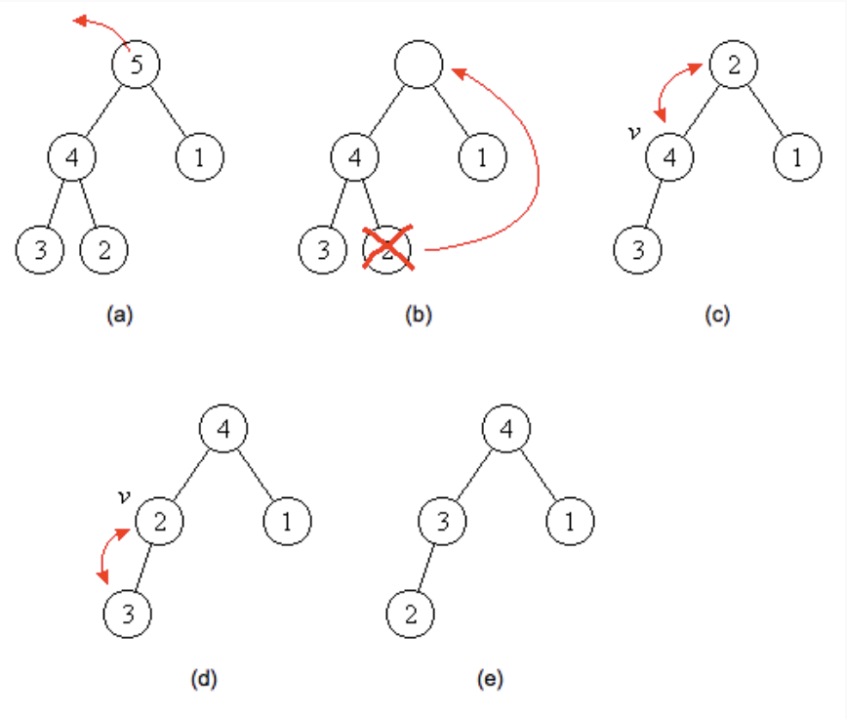
Ordenamiento por montículos (heapsort)

Tiene complejidad O(n log n). Utiliza una estructura de datos llamada heap (montículo), que se basa en la definición de un árbol binario, en el cual todos sus nodos estan ocupados por los elementos que serán ordenados. Un nodo es raiz de un montículo (heap) cuando sus nodos inmediatos tienen valores menores o iguales



Ordenamiento por montículos (heapsort)

El conjunto de elementos se estructura en un montículo (heap) con las propiedades mencionadas. Asi, el mayor elemento estará en la raiz, y se retira. Luego se lleva el elemento con la mayor profundidad a la raiz y se compara con sus vecinos para tener nuevamente el mayor valor en la raiz. Se continúa con el siguiente elemento hasta que se llega al elemento de mayor profundidad. Este algoritmo es dificil de paralelizar



Ordenamiento secuencial/paralelo

Ordenamiento	Secuencial	Paralelo (span)	Paralelo (Brent)
Bubblesort	$O(n^2)$	O(n)	$O(\frac{n^2}{p} + p)$
Bucketsort	$O(n^2)$	O(n)	$O(\frac{n^2}{p} + p)$
Mergesort	$O(n \log n)$	$O(log^2(n))$	$O(\frac{n \log n}{p} + log^2(p))$
Quicksort	$O(n \log n)$	$O(log^2(n))$	$O(\frac{n \log n}{p} + log^2(p))$
Bitonic	$O(n \log n)$	$O(log^2(n))$	$O(\frac{n \log n}{p} + log^2(p))$
Heapsort	$O(nlog(n))$	–	

- Ordenamiento en paralelo debe considerar el calculo local (n<p)
- Importante minimizar comunicación (Unidad 6: Performance)

Ordenamiento en paralelo

TABLE I: Complexity of various parallel sorting algorithms. Implicit $\mathcal{O}(\cdot)$.

Algorithm	Latency $[\alpha]$	Comm. Vol. $[\beta]$	Remarks
(All-)gather-merge	$\log p$	n	II
RFIS [4]	$\log p$	$n/\sqrt{p}$	here:robust
Bitonic [12]	$\log^2 p$	$\frac{n}{p}\log^2 p$	deterministic
Minisort [2]	$\log^2 p$	$\log^2 p$	$n = p$
HC quicksort [17]	$\log^2 p$	$\frac{n}{p}\log p$	best case
+median of medians [18]	$\log^2 p$	$p + \frac{n}{p}\log p$	average case
HC quicksort Sect. VI	$\log^2 p$	$\frac{n}{p}\log p$	here:robust
HykSort [6]	$\geq k \log_k p$	$\geq \frac{n}{p}\log_k p$	not robust
AMS [4]	$k \log_k p$	$\frac{n}{p}\log_k p$	here:robust
Sample sort [7]	$\geq p$	$\geq n/p$	+sampling cost
Multiway merges. [4], [8]	$\geq p$	$\geq n/p$	

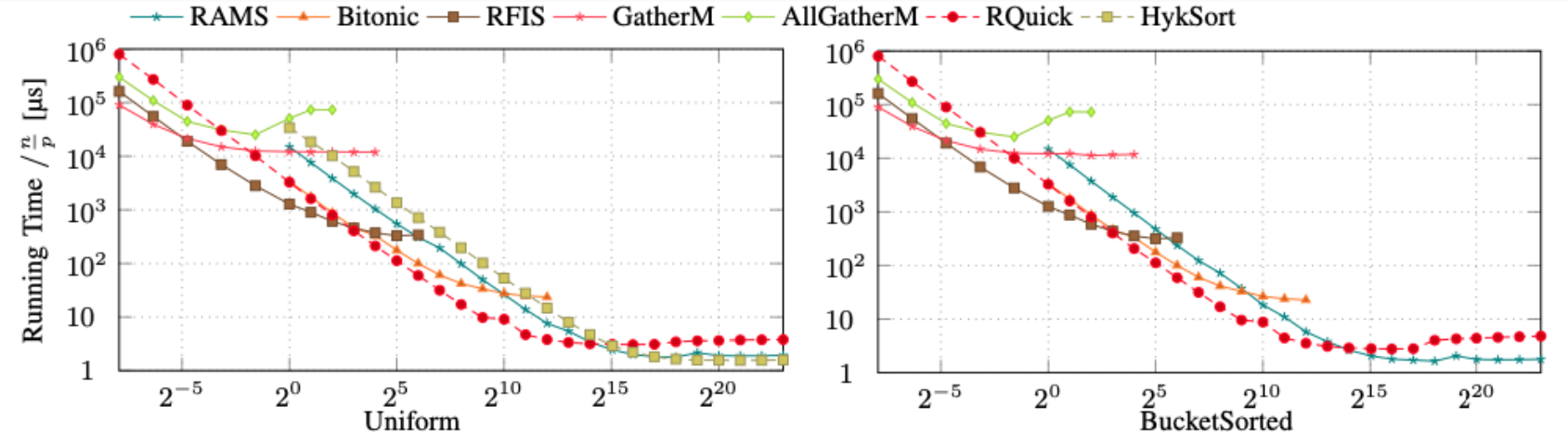


Figure 1: Complejidad y tiempo de ejecución para diversos algoritmos en 262144 núcleos, bajo condiciones iniciales distintas. De Axtmann & Sanders, 2020

Bibliografía i

- Axtmann, T., Sanders, P. et.al., *Practical Massively Parallel Sorting*, Proceedings of the 2017 IEEE International Parallel and Distributed Processing Symposium (IPDPS '17), IEEE, 2017
- Joseph Jájá *An introduction of parallel algorithms.* University of Maryland, Addison-Wesley Publishing Company, 1992
- Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP.* 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656.
- Sanders P. & Hansch J., *Efficient Massively Parallel Quicksort*, Proceedings of the 8th Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA '96), ACM, 1996

UTEC
UNIVERSIDAD DE INGENIERÍA
Y TECNOLOGÍA

